

Study Notes: Introduction to HashMap & HashTable in Java

1. The Core Problem: Why HashMaps?

In previous data structures, retrieving data took time:

- **Unsorted Arrays:** $O(N)$ (Linear Search)
- **Binary Search Trees (BST) / Heaps:** $O(\log N)$
- **Goal:** We want to retrieve elements in $O(1)$ (**Constant Time**). No traversing, no loops.

The Solution: Key-Value Pairs

HashMaps store data in Key-Value pairs (e.g., **Name -> Marks**, **Kunal -> 88**).

Instead of searching through elements, we pass the key to the HashMap, and it instantly spits out the value.

- *Real-World Applications:* Compilers pointing variable names to memory, Network Routers (IP routing), Cryptography, Virtual Memory mapping.

2. Internal Working: Hash Code & Hashing

If a HashMap is internally built on an Array, how do we use a String key (like **"Kunal"**) as an Array index?

A. The Hash Code Method

We need to convert any data type (String, Object, Float) into an integer.

- In Java, every object inherits the `hashCode()` method from the parent **Object** class.
- **"Kunal".hashCode()** applies a formula (using ASCII values, bit shifting, etc.) and returns a unique integer (e.g., **72858447**).
- *Rule:* If the value is the exact same, the hash code will ALWAYS be the exact same. (e.g., the hash code of the integer **4235678** is usually just **4235678**).

B. Hashing (Reducing the Hash Code)

Problem: The `hashCode()` gives a massive number (e.g., **72858447**), but our internal array size (**M**) might only be **10**. We cannot create an array of size 72,858,447 just to store a few items.

Solution: We use a Hash Function to reduce the hash code to fit within our array bounds.

- **Division Method:** $\text{HashValue} = \text{hashCode} \% M$ (Modulo the array size).
- *Example:* $72858447 \% 10 = 7$. The key-value pair is stored at index **7**.
- *Optimization:* Making **M** a prime number (not too close to a power of 2 or 10) provides better distribution and less predictability.

3. Collisions & Resolution Strategies

What is a Collision?

When two completely different keys yield the exact same final array index after the hashing function. (e.g., **"Kunal"** modulo 10 gives **3**, and **"Apoorv"** modulo 10 also gives **3**).

How do we fix this without overwriting data?

Strategy 1: Chaining (Separate Chaining)

Instead of storing elements directly in the array, every index of the array holds a **Linked List**.

- If a collision occurs at index 3, we simply append the new key-value pair to the end of the Linked List at index 3.
- *Worst Case Scenario*: Every single key hashes to index 3. The array is empty except for a massive Linked List at index 3. Time complexity becomes $O(N)$ for searching.

Strategy 2: Open Addressing (No Linked Lists)

Instead of chaining, we strictly allow only ONE item per slot in the array. If an index is full, we "probe" (search) for the next available empty slot.

(Note: Array size must always be $\geq$ number of elements).

1. Linear Probing:

- Formula: $\text{Hash}(\text{key}, i) = (\text{Hash}(\text{key}) + i) \% M$ (where i is the attempt number: 0, 1, 2...).
- It literally just checks the next adjacent index, then the next.
- *Problem: Clustering*. Elements clump together, making future probes take longer.

2. Double Hashing:

- Formula: $\text{Hash}(\text{key}, i) = (\text{Hash1}(\text{key}) + i * \text{Hash2}(\text{key})) \% M$
- Instead of jumping by 1, we jump by a factor determined by a *second* hash function. This spreads data out randomly and prevents clustering.

Handling Deletions in Open Addressing: You cannot simply "delete" an element and leave the slot completely empty. If you do, a probe searching for an element further down will hit the empty slot and assume the element doesn't exist. You must place a "Tombstone" or "Delete Flag" at that index.

4. Load Factor & Table Doubling (Rehashing)

To prevent chaining from becoming $O(N)$, we cheat using the **Simple Uniform Hashing Assumption**: We assume every key is equally likely to hash to any slot, independently of other keys.

The Load Factor (Alpha or α)

- $\alpha = N / M$ (Where N = Total items stored, M = Total array slots available).
- α represents the expected number of items in a single slot (the expected length of the Linked List).
- If α is $O(1)$, then HashMaps achieve $O(1)$ constant time.

Dynamic Resizing (Table Doubling):

To keep α small (usually keeping the array max 50% to 75% full):

1. Start with a small array size.
2. When N / M exceeds a threshold (e.g., 0.5 or 0.75), **Rehash**.
3. Create a completely new array that is **Double (2x)** the size.
4. Recalculate the hashes for *all* existing elements and copy them to the new array.
5. *Mathematical Proof*: Doubling the table means inserting N items takes $O(N)$ total operations across all resizes. Therefore, the cost per individual insertion is $O(N)/N = O(1)$. This is called **Amortized Constant Time**.

5. Java Implementation: The Map Interface

- **HashMap:** Not synchronized (not thread-safe). Allows ONE null key and multiple null values. Fast and preferred for standard use.
- **HashTable:** Legacy class. Synchronized (thread-safe, multiple threads won't break it). Does NOT allow any null keys/values because it relies strictly on `hashCode()` and `equals()` methods, which throw exceptions on nulls.
- **TreeMap:** Internally implemented as a Red-Black Tree. Elements are stored in **Sorted Order** based on keys. Time complexity for operations is $O(\log N)$ instead of $O(1)$.
- **HashSet:** Uses a HashMap under the hood but only stores Keys (no Values). Enforces uniqueness (ignores duplicate insertions).

6. Code Architecture: Custom HashMap Implementation (Using Chaining)

Below is the complete logic structure for building a HashMap using Generics and an Array of LinkedLists.

Step 1: The Internal Entity Class

Every entry in our map needs to hold a key and a value.

Step 2: Global Variables & Constructor

We define the Array of LinkedLists, the size tracker, and the Load Factor.

Step 3: The `put(Key, Value)` Method

1. Calculate hash, modulo by list size to get the target index.
2. Retrieve the linked list at that index.
3. Iterate through the linked list to check if the Key already exists. If it does, *update* the value and return.
4. If it doesn't exist, check the Load Factor. If $(\text{size} / \text{list.size}()) > \text{loadFactor}$, call `rehash()`.
5. Add the new `Entity` to the linked list and increment `size`.

Step 4: The `rehash()` Method

1. Save the current `list` to a variable `oldList`.
2. Re-instantiate `list` as a new ArrayList.
3. Populate `list` with double the amount of empty linked lists $(\text{oldList.size}() * 2)$.
4. Iterate over every linked list in `oldList`, and every Entity within those lists.
5. Re-insert them using our `put()` method (which will recalculate their positions for the new array size).

Step 5: The `get(Key)` and `remove(Key)` Methods

- **Get:** Calculate hash -> Go to index -> Traverse the LinkedList -> If `entity.key.equals(key)`, return `entity.value`. Else, return `null`.
- **Remove:** Calculate hash -> Go to index -> Traverse LinkedList -> If found, call `entities.remove(targetEntity)` and `size--`.

End of notes.